



CURRENT STATUS DOCUMENTATION

Elite Message Project Book

Unified business, requirements, architecture, API, database, and design documentation for the current repository state.

Version date: 2026-04-12

Document Control

Field	Value
Document name	Elite Message Current Status Project Book
Document type	Unified project documentation
Audience	Mixed team: product, management, engineering, and operations
Status scope	Reflects the checked-in repository state as of 2026-04-12
Core technologies	Next.js 16, React 19, NestJS 11, Prisma 7, MySQL, Redis, BullMQ, MinIO
Source of truth	Current codebase, schema, configuration, and docs/current-status content

Table of Contents

1 Executive Summary

1.1 Current Implementation Status

1.2 Technical Baseline

1.3 Document Intent

2 Business and Planning Documentation

2.1 Project Overview

2.1.1 Purpose

2.1.2 Business Problem

2.1.3 Product Scope Summary

2.1.4 Current Technical Baseline

2.1.5 Current Product Status

2.1.6 Target Users

2.1.7 Success Criteria

2.2 Software Proposal

2.2.1 Proposal Summary

2.2.2 Proposed Solution

2.2.3 Expected Business Outcomes

2.2.4 Why This Architecture Is Appropriate

2.2.5 Current Recommendation

2.3 Scope Document

2.3.1 Scope Statement

2.3.2 In Scope

2.3.3 Out of Scope

2.3.4 Constraints

2.3.5 Assumptions

2.3.6 Current Scope Boundary Assessment

2.4 Feasibility Study

2.4.1 Executive Assessment

2.4.2 Technical Feasibility

2.4.3 Operational Feasibility

2.4.4 Economic Feasibility

2.4.5 Schedule Feasibility

2.4.6 Legal and Security Feasibility

2.4.7 Recommendation

2.5 Budget and Timeline

2.5.1 Important Note

2.5.2 Delivery Assumptions

2.5.3 Indicative Effort

2.5.4 Indicative Budget Model

2.5.5 Suggested Milestone Plan

2.5.6 Timeline Risk Drivers

2.6 Stakeholder List

2.6.1 Important Note

2.6.2 Stakeholder Register

2.6.3 Stakeholder Priorities

2.7 Risk Register

2.7.1 Risk Scale

2.7.2 Current Risks

2.7.3 Priority Risks

3 Requirements Documentation

3.1 Business Requirements Document (BRD)

3.1.1 Purpose

3.1.2 Business Objectives

3.1.3 Business Capabilities Required

3.1.4 Business Success Measures

3.1.5 Current Business Status

3.2 Software Requirements Specification (SRS)

3.2.1 1. Scope

3.2.2 2. System Context

3.2.3 3. Users

3.2.4 4. External Interfaces

3.2.5 5. Major Software Functions

3.2.6 6. Design Constraints

3.2.7 7. Assumptions

3.2.8 8. Quality Targets

3.3 Functional Requirements

3.3.1 Identity and access

3.3.2 Workspace and tenancy

3.3.3 Instance lifecycle

- 3.3.4 Messaging
 - 3.3.5 Webhooks
 - 3.3.6 Tokens and API access
 - 3.3.7 Admin operations
 - 3.3.8 Operational support
- 3.4 Non-Functional Requirements
 - 3.4.1 Performance
 - 3.4.2 Availability and resilience
 - 3.4.3 Security
 - 3.4.4 Maintainability
 - 3.4.5 Observability
 - 3.4.6 Usability
 - 3.4.7 Portability
- 3.5 Use Cases
 - 3.5.1 UC-01 Customer signs up
 - 3.5.2 UC-02 Customer creates an instance
 - 3.5.3 UC-03 Customer links WhatsApp and uses API
 - 3.5.4 UC-04 Admin investigates a delivery problem
 - 3.5.5 UC-05 Support handles a customer issue
- 3.6 User Stories
 - 3.6.1 Customer stories
 - 3.6.2 Admin stories
 - 3.6.3 System/operator stories
- 3.7 Acceptance Criteria
 - 3.7.1 Authentication
 - 3.7.2 Workspace and instance setup
 - 3.7.3 API usage
 - 3.7.4 Messaging visibility
 - 3.7.5 Admin operations
 - 3.7.6 Operational baseline
- 4 Product and Design Documentation
 - 4.1 System Architecture Document
 - 4.1.1 Overview
 - 4.1.2 Runtime Architecture
 - 4.1.3 Architectural Characteristics

- 4.1.4 Current-State Assessment
 - 4.1.5 Supporting Infrastructure
- 4.2 High-Level Design (HLD)
 - 4.2.1 Module Decomposition
 - 4.2.2 System Boundaries
 - 4.2.3 Major Data Domains
 - 4.2.4 HLD Decisions
- 4.3 Low-Level Design (LLD)
 - 4.3.1 Customer Web
 - 4.3.2 Admin Web
 - 4.3.3 API
 - 4.3.4 Worker
 - 4.3.5 Data Layer
 - 4.3.6 Observability Layer
 - 4.3.7 LLD Observation
- 4.4 Database Schema Documentation
 - 4.4.1 Primary Technology
 - 4.4.2 Core Entity Groups
 - 4.4.3 Key Relationships
 - 4.4.4 Important Enums
 - 4.4.5 Schema Design Notes
 - 4.4.6 Important Indexes
 - 4.4.7 Reference
- 4.5 API Design and Specification
 - 4.5.1 API Style
 - 4.5.2 API Consumer Types
 - 4.5.3 Authentication Models
 - 4.5.4 Major Route Families
 - 4.5.5 Design Principles
 - 4.5.6 Current Public API Themes
 - 4.5.7 Error and Response Design
 - 4.5.8 Current-State Notes
 - 4.5.9 Reference
- 4.6 UI/UX Documentation
 - 4.6.1 Product Surfaces

4.6.2 Current UX Direction

4.6.3 Shared UI Characteristics

4.6.4 Customer UX Priorities

4.6.5 Admin UX Priorities

4.6.6 Current UX Gaps

4.7 Wireframes and Mockups

4.7.1 Note

4.7.2 Customer Signin

4.7.3 Customer Dashboard

4.7.4 Customer API Documents Page

4.7.5 Admin Control Plane

4.7.6 Mockup Guidance

4.8 Flowcharts and Sequence Diagrams

4.8.1 Customer Authentication Flow

4.8.2 Instance Creation to API Usage

4.8.3 Worker and Lifecycle Operation Sequence

4.8.4 Webhook Delivery Flow

4.8.5 Documentation Note

5 Appendix and References

5.1 Repository Structure

5.2 Local Infrastructure

5.3 Existing Reference Documents

5.4 Documentation Notes

1 Executive Summary

Elite Message is now an active multi-application platform rather than a repository scaffold. The current codebase includes:

- a customer web application for authentication, workspace operations, instance management, messaging, settings, subscription, and API onboarding
- an admin web application for users, workspaces, workers, messages, support, audit, and instance oversight
- a NestJS API that exposes customer, admin, internal, and public API surfaces
- a worker runtime that manages queued operational work and WhatsApp-linked instance execution
- a MySQL-centric data model backed by Redis, MinIO, and local observability tooling

1.1 Current Implementation Status

AREA	CURRENT IMPLEMENTATION	STATUS	CURRENT NOTE
Customer Experience	Customer authentication, dashboard routes, API docs, messages, settings, subscription, and instance detail are live.	IMPLEMENTED	The customer surface is usable end-to-end in local development.
Admin Operations	Admin authentication, MFA-capable flows, and operational explorers for users, workspaces, workers, support, audit, and messages are present.	IMPLEMENTED	The admin surface already supports day-to-day operational review and intervention.
Runtime and Messaging	Instance lifecycle records, runtime state, token handling, outbound and inbound messages, and webhook delivery tracking are implemented.	IMPLEMENTED	The schema and API flows cover the core runtime, queue, and messaging lifecycle.
API Productization	Account and instance token concepts, public API usage flows, and	IMPLEMENTED	The customer app now includes practical onboarding material for backend teams.

AREA	CURRENT IMPLEMENTATION	STATUS	CURRENT NOTE
	customer-facing code examples are implemented.		
Operational Hardening	Metrics, alerting config, rate-limit settings, signed webhooks, and retention settings exist, but hardening is not yet complete.	PARTIAL	Production-grade operational controls still need another hardening pass.
Documentation Baseline	This documentation set consolidates the current repository state and replaces earlier phase-0 assumptions.	IMPLEMENTED	The project now has a maintained current-state record in both PDF and page formats.

1.2 Technical Baseline

Area	Current choice
Monorepo	<code>pnpm</code> + <code>Turborepo</code>
Web apps	Next.js 16 + React 19
API	NestJS 11 + Express 5
Worker	BullMQ + Redis + <code>whatsapp-web.js</code>
Database	MySQL + Prisma 7
Object storage	MinIO / S3-compatible
Observability	Prometheus, Grafana, Alertmanager

1.3 Document Intent

This book is written for a mixed audience of product, engineering, and operational stakeholders. It documents the current repository state, distinguishes implemented and partial areas, and provides a standard project record across business, requirements, and product-design viewpoints.

2 Business and Planning Documentation

2.1 Project Overview

2.1.1 Purpose

Elite Message is a multi-tenant messaging platform intended to let customers create and manage WhatsApp-linked instances, send and observe messages through APIs and dashboards, and let platform administrators oversee workspaces, workers, support cases, audit trails, and operational health.

2.1.2 Business Problem

Organizations that rely on WhatsApp delivery often need:

- a managed way to operate multiple sending instances
- a customer-facing workspace for runtime control and API use
- an admin-facing control plane for operations and support
- traceability for messages, lifecycle events, tokens, and incidents

Elite Message addresses that by combining customer self-service, admin supervision, and worker-driven runtime execution in one platform.

2.1.3 Product Scope Summary

Current repository scope includes:

- customer signup, signin, Google OAuth, and session management
- admin signin and MFA-capable authentication flow
- workspace and membership management
- instance creation, lifecycle operations, runtime visibility, QR/auth state, and token management
- outbound/inbound messaging records and webhook delivery tracking
- public API usage via account and instance tokens
- support, audit, workers, and admin operational screens

2.1.4 Current Technical Baseline

- Monorepo: `pnpm` + `Turborepo`
- Web apps: Next.js 16 and React 19
- API: NestJS 11 with Express 5 and Socket.IO
- Worker: BullMQ + Redis + `whatsapp-web.js`
- Database: MySQL via Prisma 7
- Object storage: MinIO / S3-compatible
- Observability: Prometheus, Alertmanager, Grafana

2.1.5 Current Product Status

The codebase is beyond foundation stage. It already contains working customer/admin applications, API and worker services, operational data models, public API documentation UI, and local observability assets. It should be treated as an active MVP-to-post-MVP platform rather than a project bootstrap.

2.1.6 Target Users

- customer workspace owners and operators
- platform administrators
- support and operations staff
- backend integrators consuming Elite Message APIs

2.1.7 Success Criteria

- customers can authenticate, create instances, link WhatsApp, retrieve tokens, and call APIs
- admins can supervise workspaces, workers, messaging activity, support, and audit history
- runtime state, operations, and delivery history are visible and traceable
- the platform is maintainable through shared contracts, typed config, and a documented architecture

2.2 Software Proposal

2.2.1 Proposal Summary

This proposal positions Elite Message as an operational messaging platform that combines:

- customer self-service for instance creation, API access, and day-to-day runtime work
- administrator tooling for supervision, support, compliance, and intervention
- worker-based execution for QR sessions, lifecycle operations, and queue-backed message processing

2.2.2 Proposed Solution

Deliver and continue maturing a platform with these layers:

1. Customer surface
 - authentication
 - dashboard and runtime views
 - messaging explorer
 - API documentation and token use

2. Admin surface

- users, workspaces, workers, support, audit, and messaging oversight

3. Platform API

- dashboard endpoints
- admin endpoints
- internal worker endpoints
- public instance/account API endpoints

4. Worker runtime

- WhatsApp session handling
- operations execution
- queue processing
- heartbeat reporting

2.2.3 Expected Business Outcomes

- reduce manual instance handling by exposing customer self-service
- improve incident handling via centralized admin and audit tooling
- create a reusable API product for backend integrations
- establish a platform that can grow into stronger multi-tenant operational controls

2.2.4 Why This Architecture Is Appropriate

- web surfaces and runtime worker concerns are separated cleanly
- MySQL and Prisma provide a clear source of truth for multi-tenant records
- Redis and BullMQ fit queue-backed lifecycle and message processing
- Next.js + NestJS keep the stack type-safe and straightforward for a TypeScript team

2.2.5 Current Recommendation

The project should continue from its current working baseline rather than being replanned as a greenfield build. Recommended near-term focus:

- finish current-state documentation
- continue customer/admin UX refinement
- strengthen operational hardening and production-readiness gaps
- formalize release and support procedures

2.3 Scope Document

2.3.1 Scope Statement

Elite Message delivers a multi-tenant messaging control platform for customers and administrators, backed by a runtime worker and an operational API surface.

2.3.2 In Scope

2.3.2.1 Customer scope

- signup and signin
- Google OAuth signin/signup
- workspace dashboard
- instance creation and inspection
- QR/auth lifecycle visibility
- instance token visibility and rotation-assisted workflows
- outbound/inbound message exploration
- settings and subscription pages
- API documentation and language examples

2.3.2.2 Admin scope

- admin signin and MFA
- users management
- workspaces management
- workers monitoring
- instance detail inspection
- messages explorer
- support case views
- audit views

2.3.2.3 Platform scope

- MySQL-backed tenant and runtime data model
- API tokens for account and instance access
- webhook delivery tracking
- rate limiting and retention settings
- internal metrics and local observability configuration

2.3.3 Out of Scope

- native mobile applications
- dedicated billing gateway implementation
- third-party CRM or ERP connectors beyond API/webhook patterns
- full production SRE runbooks and enterprise compliance pack
- dark mode or multi-brand theming

2.3.4 Constraints

- the project is currently optimized for local/docker-based development
- some planning and commercial data is not stored in the repository

- parts of operational hardening remain partial even where configuration scaffolding exists

2.3.5 Assumptions

- MySQL remains the primary system of record
- Redis remains the queue and transient coordination layer
- customer usage centers on instance-oriented sending workflows
- admin users require higher-trust controls than customers

2.3.6 Current Scope Boundary Assessment

The repository already exceeds simple MVP scaffolding. The practical scope boundary today is not “build the first app”, but “stabilize and harden an already broad platform surface.”

2.4 Feasibility Study

2.4.1 Executive Assessment

Elite Message is technically feasible and already partially realized. The main feasibility question is no longer whether the system can be built, but whether the current implementation can be matured into a reliably operated production platform.

2.4.2 Technical Feasibility

Assessment: High

Reasons:

- the repository already contains customer web, admin web, API, worker, database schema, and observability assets
- the stack is coherent and mainstream for a TypeScript platform team
- the data model already covers users, workspaces, memberships, instances, tokens, messages, webhooks, audit, workers, and support cases

2.4.3 Operational Feasibility

Assessment: Medium

Reasons:

- local developer operations are supported through Docker Compose and environment-driven configuration
- observability configuration exists for Prometheus, Grafana, and Alertmanager
- however, production operating procedures, incident playbooks, and full hardening are not yet documented as complete

2.4.4 Economic Feasibility

Assessment: Medium to High

Reasons:

- the codebase already captures a large share of platform scope, reducing greenfield cost
- continued investment should focus on stabilization, UX refinement, and production-readiness rather than basic scaffolding
- the broad scope means uncontrolled polish/hardening work could still expand cost if not sequenced carefully

2.4.5 Schedule Feasibility

Assessment: Medium

Reasons:

- incremental delivery is feasible because the system already runs locally
- the backlog spans customer UX, admin UX, hardening, and support documentation
- final production-readiness depends on resolving operational gaps, not only feature completion

2.4.6 Legal and Security Feasibility

Assessment: Medium

Reasons:

- the system already models sessions, tokens, audit logs, webhook signing, rate limits, and admin MFA
- legal/compliance posture still needs explicit policy documentation for data retention, support handling, and incident response

2.4.7 Recommendation

Proceed with a structured stabilization program:

1. freeze and document current-state architecture and requirements
2. finish customer/admin UX refinement
3. validate operational controls end to end
4. close security, support, and observability gaps before production launch

2.5 Budget and Timeline

2.5.1 Important Note

The repository does not contain an approved commercial budget. The figures below are planning estimates derived from the current project scope and should be treated as a standard budgeting baseline, not as recorded spend.

2.5.2 Delivery Assumptions

- 1 backend/platform engineer
- 1 frontend engineer
- 0.5 QA engineer
- 0.25 product/design lead
- 0.25 DevOps/SRE support

2.5.3 Indicative Effort

Workstream	Estimated effort
Documentation and requirements baseline	1 to 2 weeks
Customer UX refinement	2 to 4 weeks
Admin UX refinement	2 to 3 weeks
Operational hardening and verification	3 to 5 weeks
Release readiness and support processes	1 to 2 weeks

Estimated total: 9 to 16 calendar weeks, depending on how much work is parallelized and how much operational hardening is required for the release target.

2.5.4 Indicative Budget Model

Using a blended delivery cost of approximately USD 2,500 to USD 5,000 per full-time-equivalent week:

- lower-bound estimate: USD 62,500
- upper-bound estimate: USD 150,000

These numbers are reasonable planning placeholders for taking the current codebase to a better-documented, better-hardened release state.

2.5.5 Suggested Milestone Plan

Milestone	Target outcome
M1	Current-state documentation baseline approved
M2	Customer UI/UX stabilization completed
M3	Admin UI/UX stabilization completed
M4	Hardening, metrics, alerts, and validation complete
M5	Release readiness review and go-live decision

2.5.6 Timeline Risk Drivers

- WhatsApp runtime reliability work
- environment and OAuth configuration issues
- backlog growth from UX redesign requests
- production hardening requirements beyond local validation

2.6 Stakeholder List

2.6.1 Important Note

The repository does not store a formal stakeholder register. The list below is a standard current-state stakeholder model inferred from the implemented product surfaces and platform responsibilities.

2.6.2 Stakeholder Register

Stakeholder	Role in project	Main interests
Product owner / sponsor	Business direction and funding	Scope, timeline, value delivery, release decisions
Customer workspace owner	Primary external operator	Instance setup, token access, messaging, visibility
Customer operator / admin	Day-to-day workspace user	Runtime status, messages, settings, API usage
Platform administrator	Internal operational owner	User/workspace governance, interventions, oversight

Stakeholder	Role in project	Main interests
Support / operations team	Issue triage and recovery	Support cases, auditability, operational tooling
Backend/platform engineering	Core system delivery	API correctness, worker orchestration, data integrity
Frontend engineering	Customer/admin UX delivery	Usability, clarity, route architecture, feedback states
QA / validation	Product quality	Regression coverage, manual flows, acceptance readiness
DevOps / SRE	Runtime reliability	Infra, metrics, alerting, deployment posture
Security / compliance reviewer	Risk control	Sessions, tokens, MFA, retention, auditability

2.6.3 Stakeholder Priorities

- customers need predictable self-service and clear API onboarding
- admins need reliable operational visibility and intervention tools
- engineering needs shared contracts, stable data models, and documented boundaries
- operations needs observability and support playbooks

2.7 Risk Register

2.7.1 Risk Scale

- Probability: Low / Medium / High
- Impact: Low / Medium / High

2.7.2 Current Risks

ID	Risk	Probability	Impact	Mitigation	Current status
R1	Product docs drift from implementation	High	Medium	Maintain current-state docs in-repo and update with major feature changes	Active

ID	Risk	Probability	Impact	Mitigation	Current status
R2	Runtime reliability issues in WhatsApp worker flows	Medium	High	Keep worker heartbeats, lifecycle event logs, and recovery tooling visible	Active
R3	OAuth or auth misconfiguration blocks access	Medium	Medium	Validate env setup, clear error messaging, and test-user instructions	Active
R4	Security controls remain partial for production use	Medium	High	Complete hardening checklist, threat review, and validation plan	Active
R5	UI redesign work expands scope without closure	High	Medium	Sequence UX changes by shell, dashboard, explorer, detail pages	Active
R6	Queue, webhook, or retry behavior becomes hard to operate at scale	Medium	High	Keep metrics, delivery explorer, replay support, and retention policies documented	Active
R7	Local-only assumptions leak into production planning	Medium	Medium	Document environment dependencies and production gaps explicitly	Active
R8	Token/session handling expectations are unclear to users	Medium	Medium	Keep API docs, token rotation guidance, and account-vs-instance token distinctions explicit	Active

2.7.3 Priority Risks

The highest-priority current risks are:

- R2 runtime reliability
- R4 incomplete production hardening
- R5 uncontrolled UX scope expansion

These should drive near-term planning decisions.

3 Requirements Documentation

3.1 Business Requirements Document (BRD)

3.1.1 Purpose

This BRD defines the business outcomes Elite Message must support based on the current platform direction and implemented product surfaces.

3.1.2 Business Objectives

- provide customers with a self-service messaging control plane
- provide administrators with an operational oversight plane
- support API-based message sending and runtime monitoring
- maintain traceability for instance lifecycle, messaging, support, and audit activities

3.1.3 Business Capabilities Required

3.1.3.1 Customer capabilities

- register and authenticate securely
- manage workspaces and instances
- link WhatsApp and monitor runtime status
- retrieve API credentials and understand how to use the API
- review outbound, inbound, and webhook-related activity

3.1.3.2 Admin capabilities

- authenticate with stronger controls than customers
- view and manage workspaces, users, instances, and workers
- inspect message and webhook activity globally
- support customers and review audit records

3.1.3.3 Platform capabilities

- persist tenant, runtime, and audit data centrally
- support tokenized account and instance API access
- process worker operations and message delivery asynchronously
- expose internal metrics and operational health

3.1.4 Business Success Measures

- customer can go from signup to first API call without admin intervention
- admin can diagnose workspace or runtime problems from the platform
- support and audit records are sufficient for issue tracing
- common operational work can be completed through the UI without database access

3.1.5 Current Business Status

The current codebase already implements much of the business surface. The remaining business risk is concentrated in polish, documentation clarity, and production-readiness rather than missing core domain concepts.

3.2 Software Requirements Specification (SRS)

3.2.1 1. Scope

Elite Message is a multi-application software system composed of customer UI, admin UI, API services, and a worker runtime for managing WhatsApp-linked messaging instances.

3.2.2 2. System Context

Primary components:

- customer web application
- admin web application
- API service
- worker service
- MySQL database
- Redis queue
- MinIO object storage
- observability stack

3.2.3 3. Users

- customer user
- platform admin
- support/operator
- backend integrator using the public API

3.2.4 4. External Interfaces

- HTTP/JSON APIs
- browser-based web UIs
- Google OAuth
- S3-compatible storage
- Prometheus-compatible metrics scraping

3.2.5 5. Major Software Functions

- authentication and session management
- workspace and membership management
- instance creation and lifecycle control

- outbound and inbound messaging data handling
- webhook delivery tracking and replay-capable operations
- support and audit workflows
- admin worker monitoring and intervention

3.2.6 6. Design Constraints

- TypeScript monorepo architecture
- MySQL as the system of record
- Next.js for both web surfaces
- NestJS API
- BullMQ/Redis for queue-backed worker operations

3.2.7 7. Assumptions

- local/docker development remains a supported primary workflow
- customer-facing APIs remain versioned under `/api/v1`
- worker runtime is required for real instance operation beyond static UI state

3.2.8 8. Quality Targets

- typed boundaries between packages
- operational traceability through audit and lifecycle records
- sufficiently clear UI flows for customer self-service
- documented security controls for tokens, sessions, and admin access

3.3 Functional Requirements

3.3.1 Identity and access

- FR-001: The system shall allow customer users to sign up with email and password.
- FR-002: The system shall allow customer users to sign in with email and password.
- FR-003: The system shall support Google OAuth for customer authentication when configured.
- FR-004: The system shall allow platform admins to sign in through a dedicated admin surface.
- FR-005: The system shall support MFA-capable admin authentication flows.

3.3.2 Workspace and tenancy

- FR-006: The system shall persist workspaces and memberships.
- FR-007: The system shall allow customer users to access only workspaces where they have membership.
- FR-008: The system shall allow admins to inspect and manage workspaces globally.
- FR-009: The system shall store workspace status and trial metadata.

3.3.3 Instance lifecycle

- FR-010: The system shall allow customer users to create instances.
- FR-011: The system shall assign each instance both an internal UUID and a public API instance ID.
- FR-012: The system shall store lifecycle status and substatus for each instance.
- FR-013: The system shall expose instance runtime state including QR-related data and timestamps.
- FR-014: The system shall support lifecycle operations such as start, stop, restart, logout, clear, takeover, and reassign.
- FR-015: The system shall record lifecycle events and operations historically.

3.3.4 Messaging

- FR-016: The system shall store outbound message records.
- FR-017: The system shall store inbound message records.
- FR-018: The system shall store outbound delivery attempts and acknowledgements.
- FR-019: The system shall support customer and admin message explorer views.
- FR-020: The system shall support public API message sending through instance-scoped routes.

3.3.5 Webhooks

- FR-021: The system shall persist webhook delivery records by event type and delivery status.
- FR-022: The system shall support signed webhook delivery semantics through instance secrets.
- FR-023: The system shall expose admin visibility into webhook deliveries.
- FR-024: The system shall support replay-capable webhook operations through the admin/API surface where implemented.

3.3.6 Tokens and API access

- FR-025: The system shall support account-level API tokens.
- FR-026: The system shall support instance-level API tokens.
- FR-027: The system shall expose token metadata without storing raw token values after creation/rotation.
- FR-028: The customer UI shall explain how to use the API with practical code examples.

3.3.7 Admin operations

- FR-029: The system shall expose admin routes for users, workspaces, workers, support, audit, messages, and instance detail.
- FR-030: The system shall store worker heartbeat data and status.

- FR-031: The system shall store support cases and assignment metadata.
- FR-032: The system shall store audit log records for major platform actions.

3.3.8 Operational support

- FR-033: The system shall expose health and metrics endpoints.
- FR-034: The system shall support rate limiting controls through configuration.
- FR-035: The system shall support retention settings for sessions, audit logs, and webhook deliveries.

3.4 Non-Functional Requirements

3.4.1 Performance

- NFR-001: The web applications should keep primary navigation responsive under normal local and staging loads.
- NFR-002: The API should support concurrent customer/admin UI access and worker traffic without blocking the request path on long-running operations.
- NFR-003: Queue-backed operations should be handled asynchronously through Redis/BullMQ rather than long-running UI requests.

3.4.2 Availability and resilience

- NFR-004: The system should degrade gracefully when external services such as Google OAuth are not configured.
- NFR-005: Worker unavailability should be visible through worker heartbeat and lifecycle status data.
- NFR-006: Health and metrics endpoints should be exposed for operational inspection.

3.4.3 Security

- NFR-007: Access tokens, refresh sessions, and API tokens must be handled securely and never expose raw secret values except at creation/rotation boundaries.
- NFR-008: Customer and admin roles must remain separated by route and API authorization checks.
- NFR-009: Admin accounts should support MFA.
- NFR-010: Public APIs should remain rate-limited and versioned.

3.4.4 Maintainability

- NFR-011: Shared contracts and typed configuration should be used across apps and packages.
- NFR-012: Core domain entities should remain centralized in the Prisma schema.
- NFR-013: Documentation should distinguish between current-state implementation and target-state planning.

3.4.5 Observability

- NFR-014: The system should emit metrics compatible with Prometheus scraping.
- NFR-015: The local development environment should support Grafana and Alertmanager configuration.
- NFR-016: Audit and lifecycle records should be retained for operational diagnosis.

3.4.6 Usability

- NFR-017: Customer workflows should prioritize clarity for signup, signin, instance setup, token retrieval, and API usage.
- NFR-018: Admin workflows should prioritize scanability for users, workspaces, workers, messages, support, and audit records.
- NFR-019: Loading, empty, error, and session-expired states should be visibly distinct and recoverable.

3.4.7 Portability

- NFR-020: The platform should run locally through Docker-based infra plus node processes.
- NFR-021: Environment configuration should remain externalized through `.env` variables.

3.5 Use Cases

3.5.1 UC-01 Customer signs up

- Primary actor: Customer user
- Preconditions: User is not yet registered
- Main flow:
 1. User opens the customer app
 2. User completes the signup form or chooses Google signup
 3. System creates the user and workspace membership baseline
 4. User reaches authenticated customer routes
- Success outcome: Customer session exists and workspace access is available

3.5.2 UC-02 Customer creates an instance

- Primary actor: Customer user
- Preconditions: Customer is authenticated and has workspace access
- Main flow:
 1. User opens the dashboard or instance creation flow
 2. User creates a new instance
 3. System persists the instance and related initial records
 4. UI exposes runtime and API access details

- Success outcome: Instance exists with internal and public identifiers

3.5.3 UC-03 Customer links WhatsApp and uses API

- Primary actor: Customer user
- Preconditions: Instance exists
- Main flow:
 1. User opens instance detail
 2. User scans the QR code / authenticates the runtime
 3. System updates runtime state and status
 4. User retrieves token guidance from the API access card or API docs page
 5. User sends a message through the public API
- Success outcome: Authenticated instance can be used through the API

3.5.4 UC-04 Admin investigates a delivery problem

- Primary actor: Platform admin
- Preconditions: Admin is authenticated
- Main flow:
 1. Admin opens messages or instance detail
 2. Admin filters outbound messages, inbound messages, or webhook deliveries
 3. Admin reviews status, attempts, and audit-related context
 4. Admin replays or escalates where supported
- Success outcome: Delivery issue is traceable and actionable

3.5.5 UC-05 Support handles a customer issue

- Primary actor: Support/admin user
- Preconditions: Support case exists or customer escalates an issue
- Main flow:
 1. Support reviews workspace, instance, support, and audit data
 2. Support identifies the failure or configuration issue
 3. Support communicates next action or resolves the issue
- Success outcome: Issue is handled using platform data rather than ad hoc investigation

3.6 User Stories

3.6.1 Customer stories

- As a customer owner, I want to sign up quickly so I can start using the platform without manual provisioning.
- As a customer user, I want to sign in with email/password or Google so I can use the access method appropriate to my account.

- As a customer operator, I want to create an instance so I can connect a WhatsApp account to the platform.
- As a customer operator, I want to see QR/auth status so I know whether the instance is ready.
- As a customer integrator, I want to see my instance ID and token so I can call the API from backend code.
- As a customer integrator, I want code examples in common languages so I can start integrating quickly.
- As a customer operator, I want to inspect outbound and inbound activity so I can diagnose message issues.

3.6.2 Admin stories

- As a platform admin, I want to sign in with stronger protection so privileged access is safer.
- As a platform admin, I want to inspect users and workspaces so I can manage platform tenants.
- As a platform admin, I want to inspect workers and instance runtime state so I can diagnose operational failures.
- As a platform admin, I want to inspect webhook deliveries and messages globally so I can trace delivery issues.
- As a platform admin, I want support and audit views so I can investigate issues with context.

3.6.3 System/operator stories

- As an operator, I want metrics and health endpoints so I can observe system state.
- As an engineer, I want typed shared contracts so changes across apps are safer.
- As an engineer, I want the schema to encode platform entities clearly so the system remains maintainable.

3.7 Acceptance Criteria

3.7.1 Authentication

- Customer user can sign up and sign in through the customer surface.
- Admin user can sign in through the admin surface.
- Google sign-in redirects correctly when OAuth credentials are configured.
- Invalid credentials produce a visible, actionable error message.

3.7.2 Workspace and instance setup

- Authenticated customer user can create an instance.
- Instance detail exposes lifecycle and runtime information.
- QR/auth state updates are visible in the UI.

- Public API instance ID is visible after instance creation.

3.7.3 API usage

- Customer user can access an API documentation page.
- The API docs page shows endpoint examples in common backend languages.
- Copy actions exist for tokens, endpoints, or snippets where exposed.
- The docs clearly distinguish workspace-level tokens from instance-level tokens.

3.7.4 Messaging visibility

- Customer messages page shows outbound and inbound activity.
- Admin messages page shows global activity and webhook context.
- Delivery status and related metadata are visible for troubleshooting.

3.7.5 Admin operations

- Admin can open workspaces, users, workers, audit, support, and instance detail routes.
- Admin views present usable filters and scanable result sets.
- Admin authentication supports MFA setup and verification where configured.

3.7.6 Operational baseline

- Local infra can be started through Docker Compose.
- API health and metrics endpoints are reachable.
- Prometheus, Grafana, and Alertmanager configs exist in the repo.
- The schema supports sessions, tokens, messages, webhook deliveries, audit logs, and support cases.

4 Product and Design Documentation

4.1 System Architecture Document

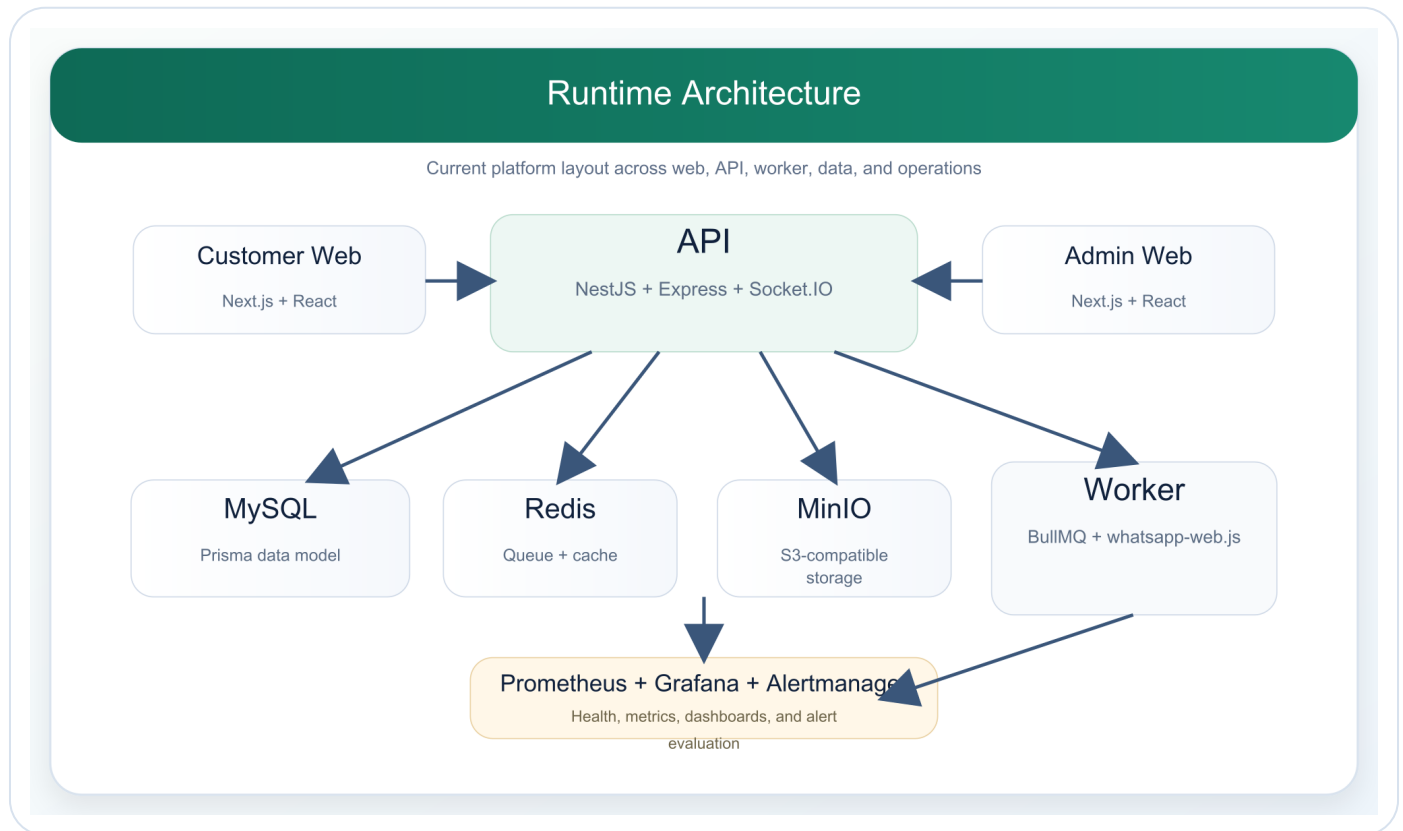
4.1.1 Overview

Elite Message is organized as a monorepo with four primary runtime applications:

- customer web
- admin web
- API
- worker

These applications depend on shared packages for configuration, contracts, UI primitives, and database access.

4.1.2 Runtime Architecture



Runtime Architecture Diagram

4.1.3 Architectural Characteristics

- multi-application, role-separated UI surfaces
- centralized API and persistence model
- queue-backed worker execution
- typed shared contracts and shared environment parsing
- local operational tooling through Docker Compose

4.1.4 Current-State Assessment

What is clearly implemented:

- customer and admin web apps
- a structured NestJS API surface
- a runtime worker with queue-backed dependencies
- a rich Prisma data model
- observability configuration assets

What remains partially mature:

- production hardening depth
- operational runbooks
- formal release documentation

4.1.5 Supporting Infrastructure

- MySQL stores tenant, runtime, audit, support, and message data
- Redis supports queue-backed and worker-related flows
- MinIO supports S3-compatible object storage needs
- Prometheus, Grafana, and Alertmanager are provisioned for local observability

4.2 High-Level Design (HLD)

4.2.1 Module Decomposition

4.2.1.1 Applications

- `apps/customer-web`
 - customer authentication
 - customer dashboard and navigation shell
 - instance detail
 - messages, settings, subscription, and API docs pages
- `apps/admin-web`
 - admin authentication
 - users, workspaces, workers, instances, support, audit, and messages pages
- `apps/api`
 - authentication
 - customer dashboard endpoints
 - admin endpoints
 - internal endpoints
 - public instance/account API endpoints
- `apps/worker`
 - worker heartbeat and health
 - instance runtime/session handling
 - queue-backed operations

4.2.1.2 Shared packages

- `packages/config` : environment parsing and logger-related configuration
- `packages/contracts` : shared Zod contracts
- `packages/db` : Prisma client and schema
- `packages/ui` : shared UI primitives and shell components

4.2.2 System Boundaries

- Web apps do not speak to the database directly.
- The worker does not own customer-facing UI concerns.
- The API is the primary integration boundary for customer UI, admin UI, and external API use.

- Shared packages keep config, schema, and UI conventions centralized.

4.2.3 Major Data Domains

- identity and sessions
- tenancy and membership
- instances and runtime
- messages and delivery
- tokens and webhook delivery
- workers and support
- auditability and retention

4.2.4 HLD Decisions

- separate admin and customer experiences instead of one overloaded application
- use workspace and instance as core business abstractions
- distinguish internal instance UUIDs from public API instance IDs
- keep long-running runtime work out of synchronous HTTP request handling

4.3 Low-Level Design (LLD)

4.3.1 Customer Web

Current route set includes:

- `/`
- `/signup`
- `/messages`
- `/settings`
- `/subscription`
- `/api-documents`
- `/instances/[id]`
- `/auth/google`

Current client concerns include:

- auth/session handling
- workspace-aware shell and topbar/sidebar navigation
- navigation progress behavior
- API documentation rendering with language examples
- instance credential handling in browser session state

4.3.2 Admin Web

Current route set includes:

- `/`
- `/messages`
- `/users`
- `/workspaces`
- `/workers`
- `/workers/[id]`
- `/instances/[id]`
- `/support`
- `/audit`

Current client concerns include:

- admin login and MFA flows
- operational explorer pages
- worker and instance inspection
- support and audit review

4.3.3 API

Current API concerns include:

- login, signup, refresh, Google auth callback, and admin MFA-related flows
- customer dashboard routes
- admin routes
- public instance/account API routes
- health, readiness, and metrics
- internal coordination endpoints

4.3.4 Worker

Current worker concerns include:

- heartbeat reporting
- runtime session state management
- queued operation execution
- WhatsApp backend integration through `whatsapp-web.js`
- local health endpoint

4.3.5 Data Layer

- Prisma schema in `packages/db/prisma/schema.prisma`
- MySQL source of truth

- relation-heavy model centered on `User` , `Workspace` , `Membership` , and `Instance`

4.3.6 Observability Layer

- Prometheus scrape config
- Alertmanager config
- Grafana dashboard provisioning

4.3.7 LLD Observation

The codebase favors clear package boundaries and shared TypeScript contracts, but some domain logic is still spread across UI clients, API endpoints, and worker behavior. That is normal at current maturity and should be documented rather than hidden.

4.4 Database Schema Documentation

4.4.1 Primary Technology

- Database: MySQL
- Access layer: Prisma 7
- Schema source: `packages/db/prisma/schema.prisma`

4.4.2 Core Entity Groups

4.4.2.1 Identity and tenancy

- `User`
- `Workspace`
- `Membership`
- `RefreshSession`
- `ApiToken`

4.4.2.2 Runtime and lifecycle

- `Instance`
- `InstanceSettings`
- `InstanceRuntimeState`
- `InstanceOperation`
- `InstanceLifecycleEvent`

4.4.2.3 Messaging and webhook delivery

- `OutboundMessage`
- `OutboundMessageAttempt`
- `InboundMessage`

- WebhookDelivery

4.4.2.4 Operations and governance

- WorkerHeartbeat
- AuditLog
- SupportCase

4.4.3 Key Relationships

- one user can have many memberships
- one workspace can have many memberships, instances, tokens, audit logs, and support cases
- one instance belongs to one workspace and can have settings, runtime state, operations, messages, webhook deliveries, lifecycle events, and tokens
- outbound messages can have many attempts and optional related webhook deliveries

4.4.4 Important Enums

- UserRole , UserStatus
- WorkspaceStatus , MembershipRole
- InstanceLifecycleStatus , InstanceOperationType , InstanceOperationStatus
- OutboundMessageType , MessageDeliveryStatus , MessageDeliveryAck
- WebhookEventType , WebhookDeliveryStatus
- WorkerStatus
- SupportCaseStatus , SupportCasePriority

4.4.5 Schema Design Notes

- API tokens are stored hashed, with prefix metadata retained for user-facing display.
- Instances have both internal UUIDs and public IDs for API use.
- Runtime state is separated from core instance metadata.
- Audit and support entities are first-class records rather than freeform logs.

4.4.6 Important Indexes

- membership uniqueness on workspace/user
- refresh session index on user and expiry
- token indexes by workspace/instance and token type
- message indexes by instance, status, schedule, reference, recipient, and sender
- webhook delivery indexes by status and retry schedule
- audit/support indexes by status, actor, entity, workspace, and instance

4.4.7 Reference

For a longer schema walkthrough, see the existing reference doc:

- [Existing database schema reference](#)

4.5 API Design and Specification

4.5.1 API Style

- HTTP/JSON
- versioned routes under `/api/v1`
- separate route families for auth, customer, admin, internal, and public instance/account APIs

4.5.2 API Consumer Types

- customer web app
- admin web app
- worker/internal services
- external backend integrators

4.5.3 Authentication Models

4.5.3.1 Dashboard session auth

- email/password login
- refresh-backed session model
- Google OAuth for customer accounts when configured
- admin MFA for privileged access

4.5.3.2 Token auth

- account-level API tokens
- instance-level API tokens
- bearer usage and query-token patterns where supported by the public API

4.5.4 Major Route Families

- `/api/v1/auth/*`
- `/api/v1/customer/*`
- `/api/v1/admin/*`
- `/api/v1/internal/*`
- `/api/v1/instance/*`
- `/health`, `/ready`, `/metrics`

4.5.5 Design Principles

- keep admin and customer concerns separated
- expose instance-scoped public APIs for sending and runtime inspection
- keep internal worker coordination separate from public routes
- preserve room for rate limiting, metrics, and retention controls

4.5.6 Current Public API Themes

- send chat and media messages
- inspect instance status
- inspect message history
- use account and instance tokens safely
- document payloads with code examples in common backend languages

4.5.7 Error and Response Design

The exact route-level shapes should be read from the implementation and existing route reference. In general, the design intent is:

- typed JSON payloads
- explicit unauthorized/forbidden handling
- rate-limit capable responses on public and dashboard routes
- machine-readable error bodies for UI and integration use

4.5.8 Current-State Notes

- API documentation exists in the customer app for practical developer onboarding.
- Admin and customer UIs rely on the same backend but different route families.
- Health and metrics endpoints exist for operational inspection.

4.5.9 Reference

For the detailed route inventory, see:

- [Existing API route list](#)

4.6 UI/UX Documentation

4.6.1 Product Surfaces

4.6.1.1 Customer surface

Current major pages:

- signin
- signup

- dashboard
- messages
- settings
- subscription
- API documents
- instance detail

4.6.1.2 Admin surface

Current major pages:

- admin signin
- dashboard
- messages
- users
- workspaces
- workers
- worker detail
- support
- audit
- instance detail

4.6.2 Current UX Direction

- light-theme-first
- role-separated customer/admin experiences
- persistent sidebar + topbar shell for authenticated routes
- scan-first explorer pages
- high-contrast operational status and action framing

4.6.3 Shared UI Characteristics

- shared `@elite-message/ui` package
- common shell, cards, badges, banners, and auth primitives
- modern auth flows with inline validation and contextual help
- progress feedback during page navigation

4.6.4 Customer UX Priorities

- quick path from authentication to instance setup
- visible instance runtime and API access
- clear distinction between workspace-level and instance-level tokens
- strong API onboarding experience for backend developers

4.6.5 Admin UX Priorities

- dense but readable operational explorers
- reliable filtering and traceability
- clear privileged-action framing
- strong session and MFA states

4.6.6 Current UX Gaps

- documentation and visual system alignment still needs ongoing maintenance
- some operational states still need more refinement for production polish
- current wireframes/mockups are mostly implemented in code rather than maintained in a separate design file

4.7 Wireframes and Mockups

4.7.1 Note

The repository currently stores implemented UI code rather than a dedicated Figma or wireframe package. The wireframes below are low-fidelity structural representations of the current product surfaces.

4.7.2 Customer Signin

+-----+-----+		
Topbar / brand		
+-----+-----+		
Brand / logo panel	Signin card	
	- email	
	- password	
	- sign in	
	- continue with Google	
	- help dialog	
+-----+-----+		

4.7.3 Customer Dashboard

+-----+-----+		
Sidebar	Sticky topbar	
- workspace	+-----+	
- dashboard	Page title / breadcrumb row	
- messages	+-----+	
- API docs	Hero / summary / signals	
- settings	+-----+	
- subscription	Work cards / tables / API access panels	
+-----+-----+		

4.7.4 Customer API Documents Page

```
+-----+
| Page header / breadcrumb |
+-----+
| Quick start summary |
+-----+
| Reference selectors: workspace, instance, language |
+-----+
| Endpoint cards + copy buttons |
+-----+
| Code examples by language |
+-----+
| Webhook examples + signature verification |
+-----+
```

4.7.5 Admin Control Plane

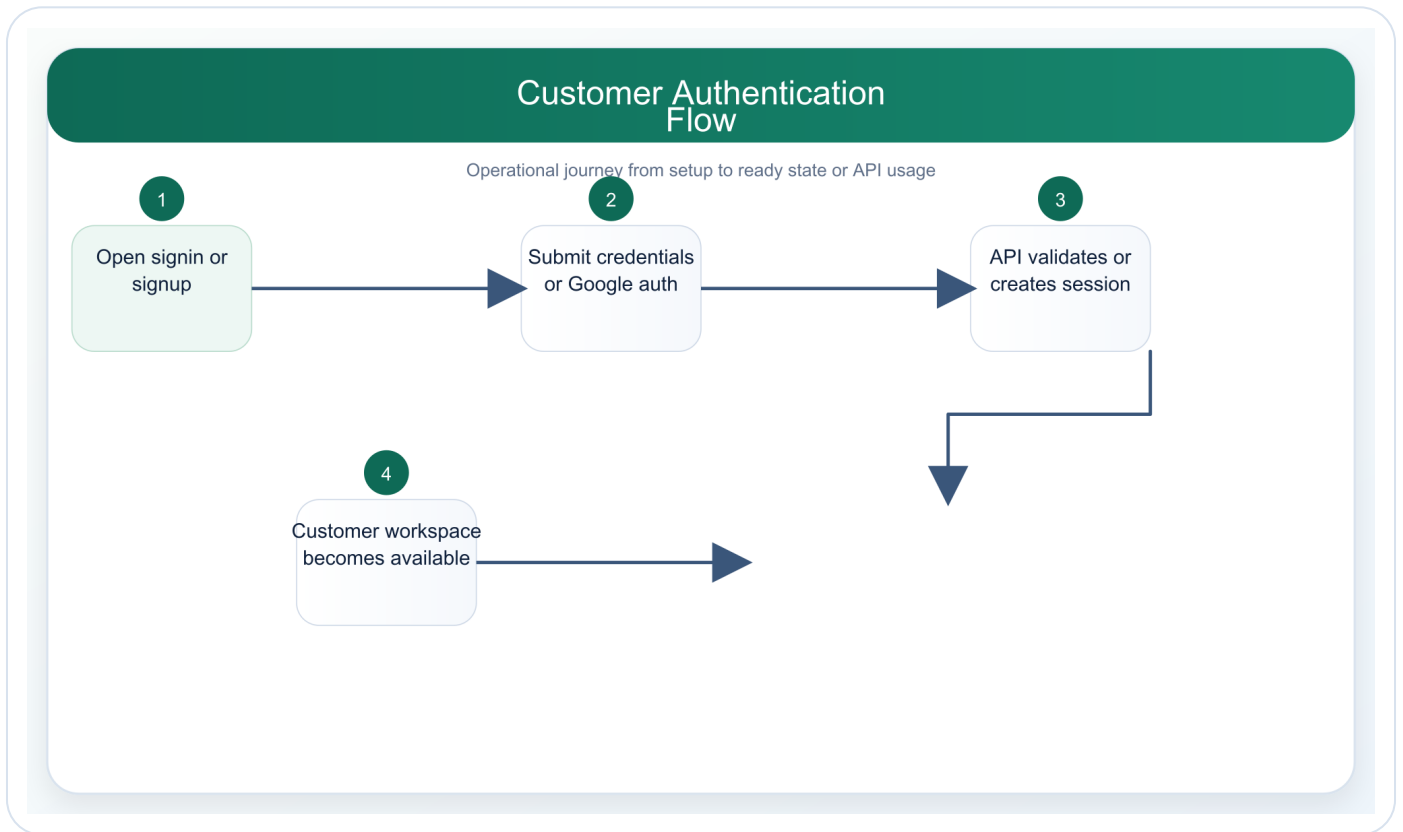
```
+-----+-----+
| Admin sidebar | Admin topbar |
| - users |
| - workspaces | Dashboard / explorer header |
| - workers |
| - messages | Tables / cards / detail content |
| - support |
| - audit |
+-----+-----+
```

4.7.6 Mockup Guidance

High-fidelity mockups should continue to reflect the implemented layout patterns already present in the Next.js apps rather than reinvent the information architecture.

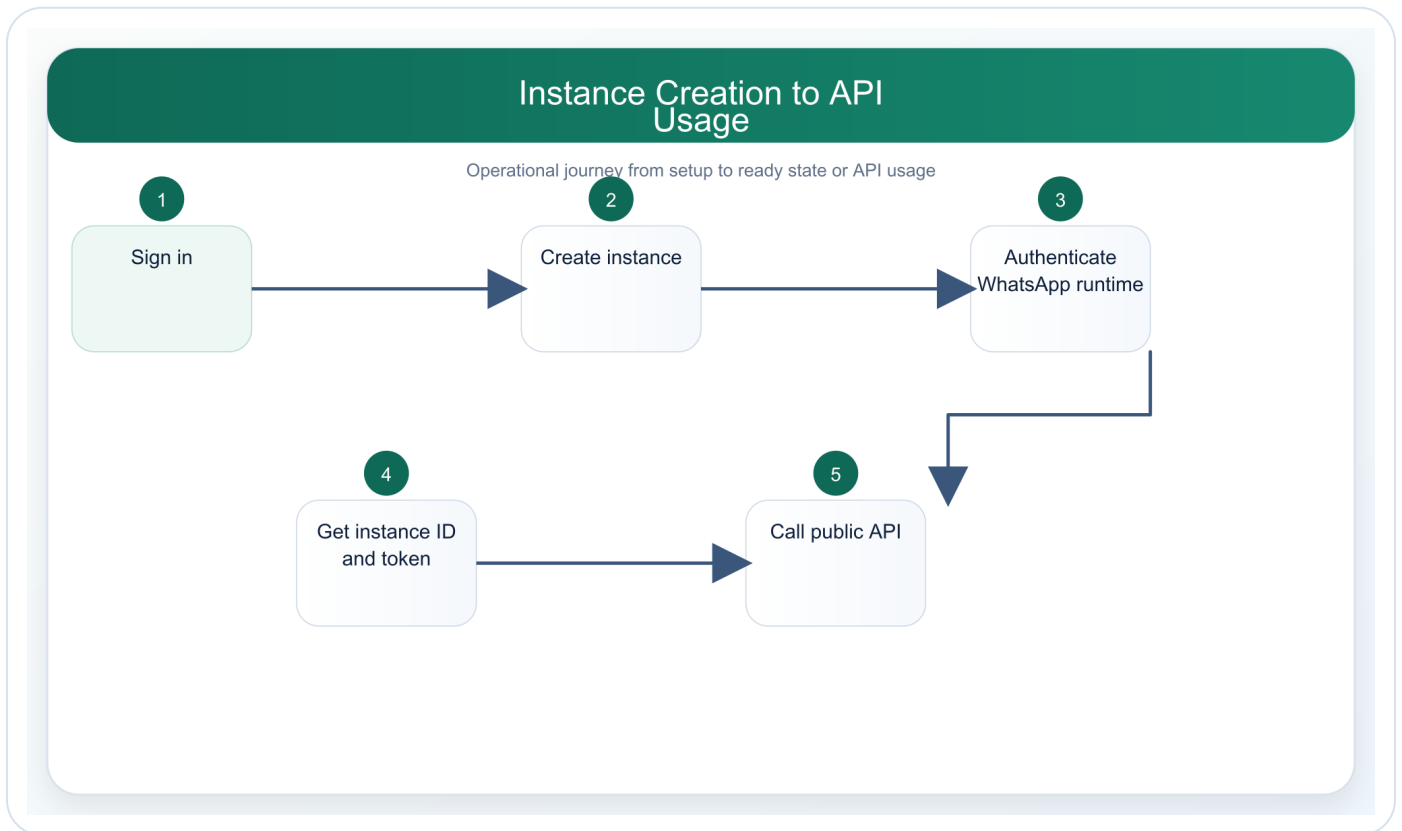
4.8 Flowcharts and Sequence Diagrams

4.8.1 Customer Authentication Flow



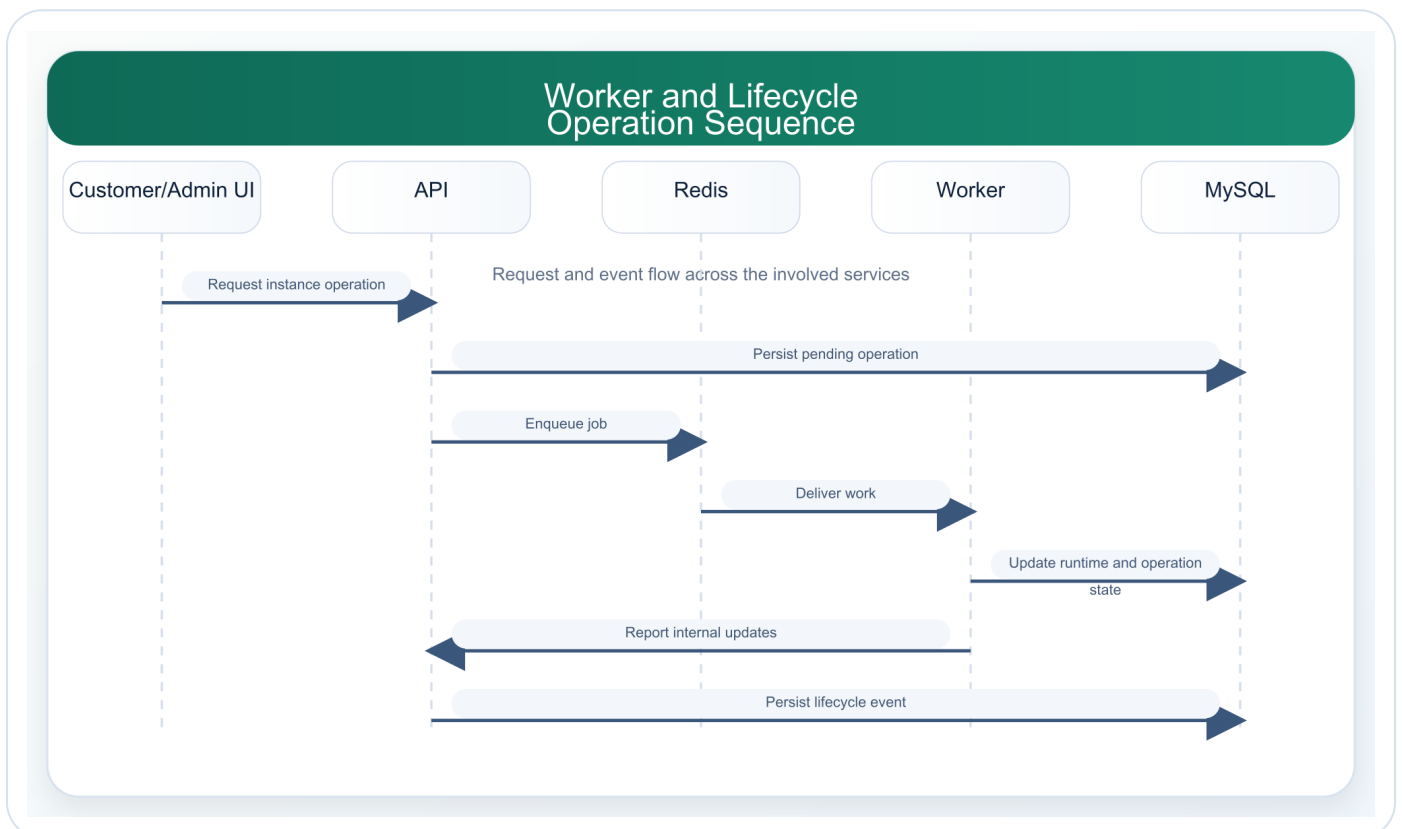
Customer Authentication Flow

4.8.2 Instance Creation to API Usage



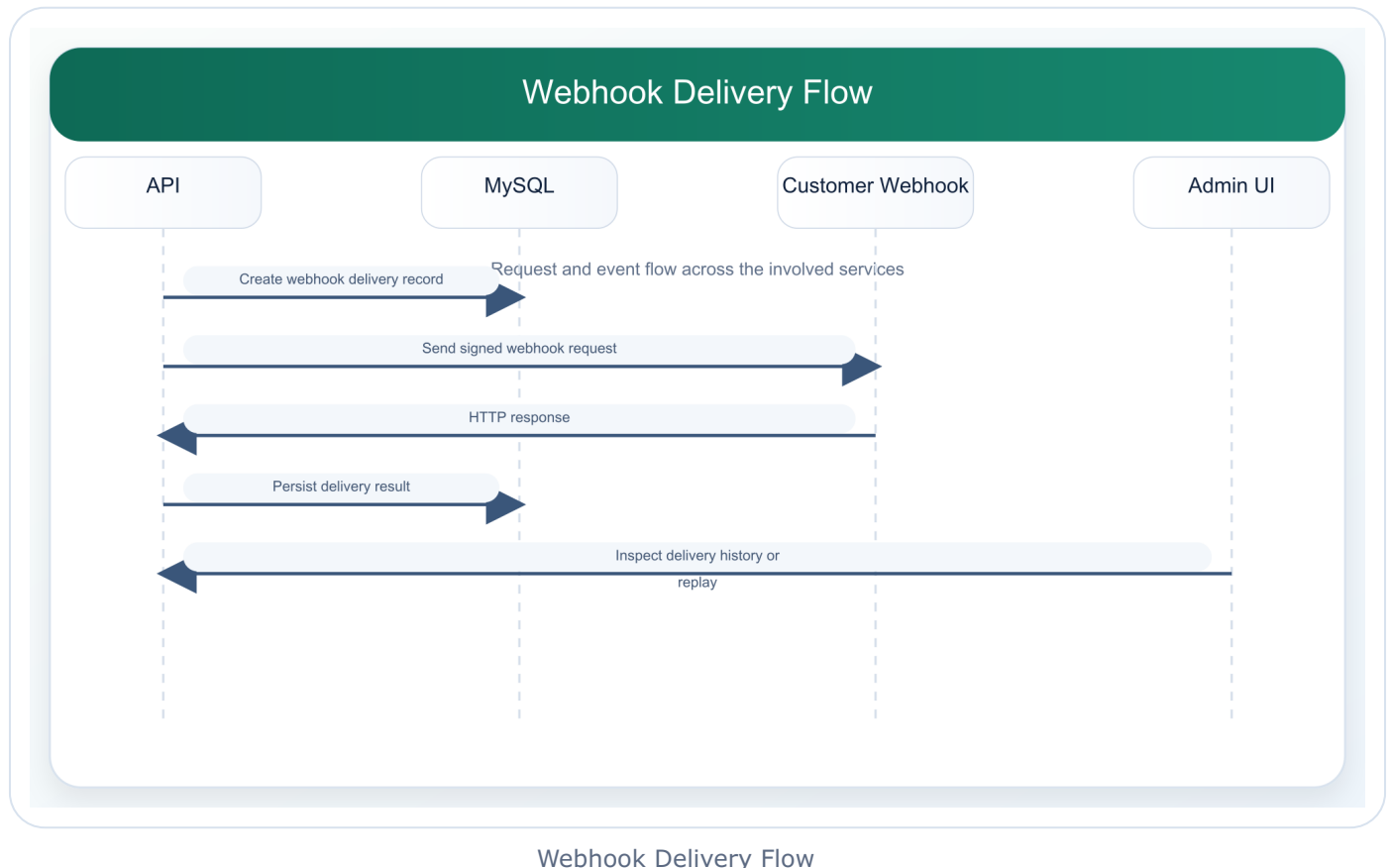
Instance Creation to API Usage

4.8.3 Worker and Lifecycle Operation Sequence



Worker and Lifecycle Operation Sequence

4.8.4 Webhook Delivery Flow



4.8.5 Documentation Note

These diagrams represent the intended current runtime relationships at a level suitable for product, architecture, and onboarding documentation. Detailed route payloads and entity fields remain documented in the API and schema references.

5 Appendix and References

5.1 Repository Structure

- apps/customer-web
- apps/admin-web
- apps/api
- apps/worker
- packages/config
- packages/contracts
- packages/db
- packages/ui

5.2 Local Infrastructure

- MySQL 8.4
- Redis 8
- MinIO
- Prometheus
- Alertmanager
- Grafana

5.3 Existing Reference Documents

- [Existing architecture reference](#)
- [Existing API route reference](#)
- [Existing database schema reference](#)
- [Existing MVP phase plan](#)

5.4 Documentation Notes

This book is a current-state documentation set prepared from the checked-in repository, package manifests, schema, route surfaces, infrastructure configuration, and implemented customer/admin experiences. Where commercial or organizational data is not stored in the repo, planning figures are documented as estimates rather than as confirmed operational records.